LAB 06 — Attention Mechanism

Môn học: Xử lý ngôn ngữ tự nhiên và ứng dụng | **Học kỳ:** Học kỳ I - 2026

Giảng viên / TA: Phạm Ngọc Hải ▪ **Hạn nộp:** 23:59 - 28/10/2026

Chủ đề: From Recurrent State to Context-Aware Representation

1. Bối cảnh

Ở LAB 05, sinh viên đã chuyển từ static representation sang sequence modeling:

```
Text
 ↓
Token sequence
 ↓
Embedding
 ↓
RNN / LSTM / GRU
 ↓
Hidden states
 ↓
Classification
```

Sequence model có khả năng sử dụng thứ tự của token và duy trì hidden state theo thời gian.

Tuy nhiên, một vấn đề xuất hiện khi sequence dài.

Ví dụ:

```
"The movie was extremely long and
the acting was sometimes boring,
but the final scene was absolutely fantastic."
```

Nếu model cần xác định sentiment của toàn bộ câu, thông tin quan trọng có thể nằm ở một vị trí rất xa.

Trong recurrent model:

```
h1 → h2 → h3 → ... → hT
```

thông tin phải đi qua nhiều bước.

LAB 06 đưa vào một cơ chế mới:

```
Query
  │
  ├── Key
  │
  └── Value
        ↓
   Attention weights
        ↓
 Context vector
```

Ý tưởng trung tâm:

> **Thay vì buộc model nén toàn bộ sequence vào một hidden state duy nhất, attention cho phép model tập trung vào những phần của sequence liên quan hơn tới thông tin đang cần.**

---

2. Mục tiêu học tập

Sau khi hoàn thành lab, sinh viên có thể:

- **LO1:** Giải thích motivation của attention.
- **LO2:** Tính attention score giữa query và key.
- **LO3:** Tính attention weights bằng softmax.
- **LO4:** Tính context vector từ attention weights và values.
- **LO5:** Giải thích vai trò của Query, Key và Value.
- **LO6:** Phân biệt dot-product attention và scaled dot-product attention.
- **LO7:** Implement attention từ đầu bằng NumPy.
- **LO8:** Visualize attention weights.
- **LO9:** Phân tích ảnh hưởng của temperature/scaling.
- **LO10:** So sánh recurrent representation với attention-based representation.
- **LO11:** Nhận diện giới hạn của attention.
- **LO12:** Hiểu attention như bước chuyển tiếp tới Transformer.

---

3. Cấu trúc lab

Giai đoạn	Thời lượng	Hoạt động
Theory recap	20 phút	RNN limitation và attention
Calculation	30 phút	Scores, softmax, context
Prediction	10 phút	Attention behavior
Experiment 1	20 phút	Dot-product attention
Core implementation	35 phút	Attention từ NumPy
Experiment 2	25 phút	Scaling và temperature
Experiment 3	25 phút	Attention visualization
Application	25 phút	Context-aware classification
Error analysis	20 phút	Attention patterns
Reflection	10 phút	Attention → Transformer

---

4. Theory recap

4.1 Từ hidden state sang context vector

Ở LAB 05, một cách đơn giản để classification là sử dụng hidden state cuối:

```
x1 → x2 → x3 → ... → xT
                 ↓
                hT
                 ↓
            classifier
```

Vấn đề:

Liệu (h_T) có phải luôn là representation tốt nhất của toàn bộ sequence?

Attention đưa ra một cách khác.

Thay vì chỉ sử dụng:

$$h_T,$$

model có thể xem xét:

$$h_1, h_2, \dots, h_T$$

và quyết định hidden state nào quan trọng hơn.

```
h1 ──┐
h2 ──┤
h3 ──┤→ Attention → Context
h4 ──┤
h5 ──┘
```

---

5. Attention score

Cho query:

$$q$$

và các key:

$$k_1, k_2, \dots, k_T.$$

Dot-product score:

$$s_i = q^T k_i.$$

Score càng lớn nghĩa là query và key càng tương đồng theo không gian biểu diễn.
Ví dụ:

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

và:

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Khi đó:

$$s_1 = q^T k_1 = 1$$

và:

$$s_2 = q^T k_2 = 0.$$

Do đó query có xu hướng chú ý tới (k_1) nhiều hơn (k_2).

---

6. Softmax attention weights

Các scores được chuyển thành probability-like weights:

$$\alpha_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)}.$$

Do đó:

$$\sum_i \alpha_i = 1.$$

Ví dụ:

$$s = [2, 1, 0].$$

Sinh viên tính:

$$\alpha = \text{softmax}(s).$$

Sau đó trả lời:

> Token nào nhận attention lớn nhất?

---

7. Context vector

Sau khi có attention weights:

$$\alpha_1, \alpha_2, \dots, \alpha_T,$$

context vector được tính:

$$c = \sum_i \alpha_i v_i.$$

Trong đó (v_i) là value tương ứng với key (k_i).

Toàn bộ quá trình:

```
Query
  │
  ├── dot(K)
  ↓
Scores
  │
  ├── softmax
  ↓
Attention weights
  │
  ├── weighted sum(V)
  ↓
Context vector
```

Đây là computation cốt lõi của attention.

---

8. Bài tập tính toán

Bài 1 — Attention scores

Cho:

$$q = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

và:

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Tính:

$$s_i = q^T k_i.$$

Viết toàn bộ score vector.

---

9. Bài 2 — Softmax

Cho:

$$s = [1, 2, 3].$$

Tính:

$$\alpha = \text{softmax}(s).$$

Sinh viên phải chỉ ra:

$$\sum_i \alpha_i = 1.$$

Sau đó trả lời:

Vì sao phần tử có score lớn nhất nhận weight lớn nhất?

---

10. Bài 3 — Context vector

Cho:

$$\alpha = [0.1, 0.2, 0.7].$$

Các value:

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

$$v_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

$$v_3 = \begin{bmatrix} 2 \\ 2 \end{bmatrix}.$$

Tính:

$$c = 0.1v_1 + 0.2v_2 + 0.7v_3.$$

Sau đó giải thích:

Value nào đóng góp nhiều nhất vào context vector?

---

11. Bài 4 — Toàn bộ attention computation

Cho:

$$q = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

Keys:

$$k_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad k_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad k_3 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Values:

$$v_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 0 \\ 2 \end{bmatrix}, \quad v_3 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Thực hiện toàn bộ:

```
q
 ↓
scores
 ↓
softmax
 ↓
attention weights
 ↓
weighted sum
 ↓
context vector
```

Không được sử dụng thư viện attention trong phần calculation.

---

12. Prediction trước experiment

Trước khi chạy code:

```
Prediction:
Reason:
Confidence:
```

Prediction 1

Nếu một key có dot product lớn hơn các key khác, attention weight của nó sẽ như thế nào?

Prediction 2

Nếu tất cả scores giống nhau:

$$s = [1, 1, 1, 1],$$

attention weights sẽ như thế nào?

Prediction 3

Nếu một score lớn hơn rất nhiều:

$$s = [1, 1, 10],$$

attention sẽ tập trung như thế nào?

Prediction 4

Nếu sequence tăng từ:

```
10 tokens → 1000 tokens
```

chi phí tính attention sẽ thay đổi như thế nào?

Prediction 5

Attention có luôn chọn đúng token mà con người cho là quan trọng nhất không?

---

13. Scaled dot-product attention

Dot-product attention có thể tạo ra score rất lớn khi vector dimension tăng.
Do đó sử dụng:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Trong đó:

$$d_k$$

là dimension của key.

Quy trình:

```
QKᵀ
 ↓
divide √dₖ
 ↓
softmax
 ↓
weights
 ↓
V
```

---

14. Bài tập — Scaling

Cho:

$$d_k = 4.$$

và:

$$QK^T = \begin{bmatrix} 4 & 2 & 0 \end{bmatrix}.$$

Tính scaled scores:

$$\frac{QK^T}{\sqrt{4}}.$$

Sau đó tính softmax.

So sánh với trường hợp không scaling.

Trả lời:

> Scaling làm thay đổi độ tập trung của softmax như thế nào?

---

15. Temperature

Để quan sát rõ hơn behavior của softmax, sử dụng:

$$\text{softmax}\left(\frac{s}{T}\right).$$

Trong đó (T) là temperature.

Thử:

```
T = 0.5
T = 1.0
T = 2.0
```

với cùng score vector.

Ghi:

Temperature	Max weight	Entropy	Nhận xét
0.5			
1.0			
2.0			

Mục tiêu:

> Quan sát trực tiếp việc temperature thấp làm distribution tập trung hơn và temperature cao làm distribution phẳng hơn.

---

16. Core implementation

Sinh viên tự implement scaled dot-product attention bằng NumPy.

Yêu cầu:

```
def scaled_dot_product_attention(Q, K, V):
    ...
```

Thực hiện:

$$S = \frac{QK^T}{\sqrt{d_k}}$$

sau đó:

$$A = \text{softmax}(S)$$

và:

$$C = AV.$$

Hàm phải trả về:

```
scores
attention_weights
context
```

Không được gọi trực tiếp:

```
torch.nn.MultiheadAttention
```

hoặc:

```
keras.layers.Attention
```

trong phần core implementation.

---

17. Numerical stability

Sinh viên phải implement softmax ổn định về số:

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}}.$$

Không nên tính trực tiếp:

```
np.exp(x)
```

với score rất lớn mà không xử lý.

Test với:

$$x = [1000, 1001, 1002].$$

Trả lời:

> Tại sao phép trừ `max(x)` không làm thay đổi kết quả softmax?

---

18. Experiment 1 — Attention distribution

Tạo một sequence:

```
"The movie was surprisingly good despite
a very slow beginning."
```

Gán một query giả lập.

Tính attention weights.

Hiển thị:

Token	Score	Attention
The		
movie		
was		
surprisingly		
good		
despite		
a		
very		
slow		
beginning		

Vẽ attention:

```
Token
  ↑
  |      ▌
  |      ▌
```



Không yêu cầu attention phải "đẹp".

Mục tiêu là quan sát:

Model đang phân phối trọng số như thế nào?

---

19. Experiment 2 — Query thay đổi

Sử dụng cùng một sequence nhưng thay đổi query.

Ví dụ:

```
Query A → sentiment
Query B → topic
```

Sinh viên quan sát:

Attention distribution có thay đổi khi query thay đổi không?

Điểm quan trọng:

Attention không đơn giản là một danh sách "từ quan trọng nhất" cố định cho cả câu.

Nó phụ thuộc vào query.

---

20. Self-attention

Ở attention tổng quát:

```
Query
  ↓
Key / Value
```

Query có thể đến từ một nguồn khác với Key/Value.

Trong **self-attention**, Q, K và V được tạo từ cùng một sequence:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V.$$

Sau đó:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Do đó:

```
X
│
├── WQ → Q
├── WK → K
└── WV → V
       ↓
   Self-Attention
       ↓
 Contextual representations
```

---

21. Bài tập self-attention

Cho sequence gồm 3 token:

```
["I", "like", "cats"]
```

Giả sử embedding matrix:

$$X \in \mathbb{R}^{3\times 2}.$$

Sinh viên được cung cấp các ma trận:

$$W_Q, \quad W_K, \quad W_V.$$

Tính:

$$Q = XW_Q$$

$$K = XW_K$$

$$V = XW_V.$$

Sau đó tính:

$$QK^T.$$

Tiếp tục:

$$softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V.$$

Mục tiêu là để sinh viên nhìn thấy:

> Một token có thể tương tác với các token khác trong cùng sequence.

---

22. Attention matrix

Với sequence gồm (T) tokens:

$$A \in \mathbb{R}^{T \times T}.$$

Mỗi hàng biểu diễn:

> Token hiện tại đang attention tới các token nào?

Ví dụ:

```
              I    like   cats
I           0.6   0.2    0.2
like        0.1   0.7    0.2
cats        0.1   0.3    0.6
```

Sinh viên phải visualise attention matrix bằng heatmap.

---

23. Experiment 3 — Attention visualization

Sử dụng một số câu có quan hệ ngữ nghĩa rõ:

```
"The cat that I adopted yesterday is sleeping."
```

và:

```
"The book that I bought yesterday was expensive."
```

Visualize attention matrix.

Sau đó chọn 3 token:

```
cat
adopted
sleeping
```

và quan sát các token mà chúng attention tới.

Không yêu cầu sinh viên khẳng định rằng heatmap chứng minh model "hiểu ngữ pháp".

Phải phân biệt:

```
attention pattern
```

với:

```
linguistic explanation
```

Attention visualization là bằng chứng về **phân phối attention**, không tự động là bằng chứng về reasoning hay linguistic understanding.

---

24. Experiment 4 — RNN context vs Attention context

Quay lại bài toán sentiment ở LAB 05.

So sánh:

```
RNN/LSTM
 ↓
final hidden state
 ↓
classifier
```

với:

```
Embedding
 ↓
Self-Attention
 ↓
context representation
 ↓
classifier
```

Bảng:

Model	Accuracy	F1	Training time	Parameters
LSTM				
Attention				

Không yêu cầu Attention phải thắng LSTM.

Câu hỏi chính:

> Representation của hai model khác nhau ở đâu?

---

25. Experiment 5 — Sequence length và computational cost

Một attention matrix có kích thước:

$$T \times T.$$

Do đó số phần tử tăng theo:

$$O(T^2).$$

So sánh:

Sequence length	Attention matrix
10	(10×10)
100	(100×100)
500	(500×500)
1000	(1000×1000)

Tính số phần tử.

Sau đó trả lời:

> Tại sao self-attention có vấn đề computational cost khi sequence rất dài?

Đây là một limitation quan trọng mà sinh viên phải biết trước khi sang Transformer.

---

26. Application — Context-aware sentiment

Xây dựng một classifier sử dụng attention.

Pipeline:

```
Raw text
 ↓
Tokenizer
 ↓
Embedding
 ↓
Sequence representation
 ↓
Self-Attention
 ↓
Context vector
 ↓
Classifier
 ↓
Sentiment
```

Thử các câu:

```
"I like this movie."

"I do not like this movie."

"I thought this movie would be bad,
but it was actually excellent."
```

Với mỗi câu:

- prediction;
- probability;
- attention visualization.

Sinh viên chọn ít nhất một prediction đúng và một prediction sai để phân tích.

---

27. Error analysis

Chọn:

```
3 correct
3 incorrect
```

Phân loại lỗi:

```
negation
long-range dependency
ambiguous wording
sarcasm
rare word
domain shift
truncation
noise
```

Với attention-based model, bổ sung:

> Attention có tập trung vào token mà sinh viên dự đoán là quan trọng không?

Nếu không:

Điều này có thể giải thích prediction sai không?

Không được kết luận:

"Attention nhìn sai từ nên model sai."

Phải trình bày đây là một **hypothesis**, sau đó đối chiếu với prediction và model behavior.

---

28. So sánh LAB 05 và LAB 06

Property	RNN/LSTM/GRU	Attention
Sequence order	v	v
Hidden state	v	Không bắt buộc
Sequential computation	v	Không bắt buộc
Direct token-to-token interaction	Limited	v
Long-range interaction	Difficult	Easier
Parallel computation	Limited	v
Cost theo sequence length	Gần tuyến tính theo timestep	(O(T^2)) cho full self-attention
Context representation	Recurrent state	Weighted combination

Câu hỏi:

Attention giải quyết vấn đề gì của recurrent processing?

Và:

Attention tạo ra vấn đề mới nào?

---

29. Một experiment quan trọng — Attention không miễn phí

Cho:

$$T = 1000.$$

Attention matrix có:

$$1000^2 = 1,000,000$$

phần tử.

Nếu:

$$T = 10,000,$$

thì:

$$10,000^2 = 100,000,000.$$

Sinh viên phải nhận ra:

> Attention giải quyết một số vấn đề của RNN nhưng tạo ra computational cost tăng theo chiều dài sequence.

Đây là một điểm rất quan trọng để tránh hình thành quan niệm sai:

> "Attention tốt hơn RNN nên luôn thay thế RNN."

---

30. AI policy

AI được phép sử dụng cho:

- debugging;
- documentation;
- visualization;
- NumPy/PyTorch API;
- plotting;
- giải thích runtime errors;
- boilerplate code.

AI không được sử dụng để thay thế:

- calculation;
- prediction trước experiment;
- giải thích Query/Key/Value;
- tính attention matrix;
- interpretation của attention visualization;
- error analysis;
- reflection;
- individual learning check.

Nếu AI sinh code:

```
AI assistance statement

Tool:
Purpose:
Generated:
Modified:
Verified:
```

Sinh viên phải có khả năng giải thích:

```
Q
K
V
QKᵀ
softmax
AV
```

---

31. Individual learning check

Giảng viên random một câu trong 3 phút.

Ví dụ:

> Attention score được tính như thế nào?

hoặc:

> Tại sao phải dùng softmax?

hoặc:

Query, Key và Value có vai trò gì?

hoặc:

Context vector được tính như thế nào?

hoặc:

Tại sao scaled dot-product attention chia cho $\sqrt{d_k}$?

hoặc:

Self-attention khác attention thông thường ở đâu?

hoặc:

Nếu sequence dài gấp đôi thì attention matrix thay đổi như thế nào?

hoặc:

Attention có chứng minh model đang reasoning không?

hoặc:

Tại sao attention có thể xử lý long-range interaction tốt hơn recurrent state?

---

32. Deliverables

```
lab06/
├── README.md
├── calculations.md
├── prediction.md
├── attention_numpy.py
├── self_attention.ipynb
├── results.csv
├── attention_heatmap.png
├── confusion_matrix.png
├── error_analysis.md
└── reflection.md
```

---

33. Rubric

Thành phần	Điểm
Theory & calculation	15
Prediction	10
Attention implementation	20
Self-attention implementation	10
Scaling / temperature experiment	10
Attention visualization	10
Application	10
Quantitative evaluation	5
Error analysis	5
Reflection	5
Tổng	**100**

---

34. Peer assessment

Reviewer kiểm tra:

```
Theory
[ ] Attention score được giải thích
[ ] Softmax được giải thích
[ ] Query / Key / Value được phân biệt
[ ] Scaling được giải thích

Calculation
[ ] Scores đúng
[ ] Softmax đúng
[ ] Context vector đúng

Implementation
[ ] Attention được implement
[ ] Numerical stability được xử lý
[ ] Self-attention được implement

Experiment
[ ] Có temperature experiment
[ ] Có attention visualization
[ ] Có sequence-length analysis
[ ] Có quantitative evaluation

Analysis
[ ] Ít nhất 3 lỗi được phân tích
[ ] Attention pattern được mô tả
[ ] Không đồng nhất attention với reasoning

Understanding
[ ] Giải thích được Q/K/V
[ ] Giải thích được \(QK^T\)
[ ] Giải thích được softmax
[ ] Giải thích được \(O(T^2)\)
```

---

35. Mạch nối LAB 05 → LAB 06

LAB 05:

```
Sequence
 ↓
RNN / LSTM / GRU
 ↓
Hidden state
 ↓
Classification
```

LAB 06:

```
Sequence
 ↓
Q / K / V
 ↓
Attention
 ↓
Context representation
 ↓
Classification
```

LAB 05 đặt câu hỏi:

Làm thế nào để duy trì thông tin qua sequence?

LAB 06 đặt câu hỏi:

Làm thế nào để chọn thông tin liên quan trong sequence?

Attention cho phép:

$$\boxed{c_i = \sum_j \alpha_{ij} v_j}$$

với:

$$\alpha_{ij} = softmax\left(\frac{q_i^T k_j}{\sqrt{d_k}}\right).$$

Đây là bước chuyển quan trọng:

```
RNN
 ↓
Recurrent state
```

sang:

```
Attention
 ↓
Direct interaction between positions
```

Nhưng LAB 06 cũng phải để lại hai câu hỏi:

Làm thế nào để thực hiện attention cho toàn bộ sequence một cách hiệu quả?

và:

Nếu mỗi token có thể attention tới mọi token khác, làm thế nào xây dựng một architecture hoàn chỉnh để xử lý sequence?

Đó là điểm xuất phát của LAB 07:

```
LAB 07

Self-Attention
      ↓
Multi-Head Attention
      ↓
Positional Encoding
      ↓
Transformer Block
      ↓
Transformer
```

LAB 06 vì vậy **không dạy Multi-Head Attention sâu**. Việc dùng ngay Q/K/V, multi-head, positional encoding và full Transformer vào cùng một lab hôm nay, các em rất dễ nhớ API nhưng không hiểu attention mechanism. LAB 06 này giúp cho các em **tính được attention bằng tay và tự implement được một attention layer đơn giản**; LAB 07 mới ghép các thành phần đó thành Transformer.